

第三节

Agent OS 是什么，如何设计实现一个 Agent OS

Agent OS 是 2026 年企业级 AI 落地的标配，也是训练营最重的项目。这节课用五组件拆解 Agent 的技术本质，对比主流开源项目，最后讲清楚 OryxOS 怎么设计、为什么这么设计。

《企业级AI编程实战营》· 技术深度展示



Agent OS 是 2026 年企业 AI 落地的新基础设施

企业要同时跑 N 个 Agent, 这 N 个 Agent 需要共享一套底座

为什么现在?

MCP 在 2024 年底成为全行业共识标准, 工具和运行时彻底解耦。Agent OS 有了可以站立的地基。

为什么是底座?

渠道接入、LLM 路由、记忆管理、工具调用、安全审计--这些能力每个 Agent 都需要, 都建一遍是浪费, 放进底座才是工程。

业界在做什么?

OpenClaw (247k stars, Node.js)、Hermes Agent (135k stars, Python) 已经验证了这个方向。但 Java 生态, 真空。

一个 Agent OS 装好, 所有 Agent 上线只需要写 Profile + Tool, 不用每次重新造底座。

Agent 在技术上到底是什么：五组件

任何 Agent，拆开都是这五件事的组合

LLM

大脑

接受文本输入，给出文本输出。推理、规划、生成、决策——全靠 LLM。OryxOS 通过 Provider 抽象支持 12 个主流 LLM，运行时切换无 lock-in。

Memory

记忆

跨对话保留状态。三层：会话记忆（当前对话）+ 长期记忆（MEMORY.md）+ 情景记忆（任务过程）。没有 Memory，Agent 每次都从零开始，不是助手，是工具。

Loop

驱动引擎

ReAct 循环：Reason（LLM 思考）→ Act（工具执行）→ Observe（结果回填）→ 继续。这个循环让 Agent 能自主完成多步骤任务，不是一问一答。

Tools

执行能力

Agent 通过 LLM Function Calling 决定调哪个工具。OryxOS 内置 5 个 Tool，业务方通过 SKILL.md + MCP 零代码扩展。工具决定 Agent 能干什么。

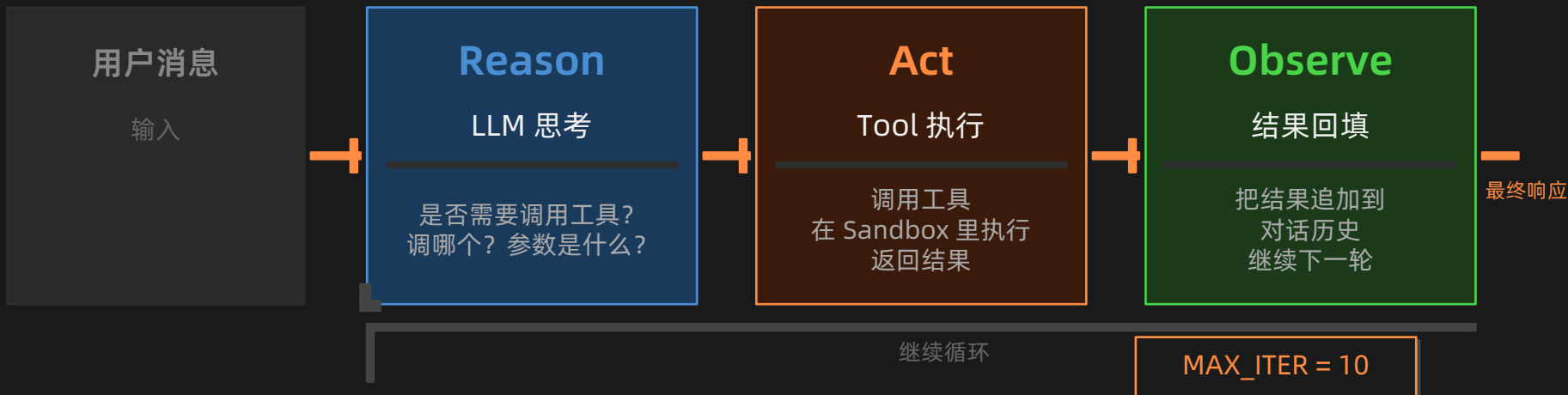
Environment

运行环境

Sandbox 隔离保证安全边界，工作区 .oryxos/ 管理配置和数据，Channel 决定 Agent 怎么被触发。Environment 是 Agent 存在的容器。

ReAct Loop: Agent 的大脑

Reason + Act = ReAct · 这是所有 Agent 框架的底层机制，OryxOS 自己实现，不超过 100 行 Java。



Session 消息累积

每一轮的 LLM 响应 + Tool 结果都追加到 Session 对话历史。完整调用链对外可查可审计。历史超过 LLM context window 时截断保留近期。

ReAct 不是魔法，是一个清晰的循环。任何 Agent 行为都能从这个循环里推导出来

五组件组合起来：一个运维告警场景

"凌晨 3 点告警：service-api 响应超时 → 自动分析 → 自动重启 → 企业微信通知"

01 Environment Webhook 进来，Channel 把告警消息路由到运维 Agent（Profile 配置：ops-agent）。

03 Tools Act：调用 log_query Tool 查 service-api 最近 10 分钟错误日志，Sandbox 检查通过，执行。

05 Loop 继续循环：调用 restart_service Tool 执行重启，等待健康检查通过。

02 Loop ReAct 循环启动。LLM 拿到告警消息，思考：需要先查日志。

04 Memory Observe：日志追加到 Session。LLM 看到 OOM 错误，查 MEMORY.md 发现这是已知 bug，有 mitigation。

06 Tools 调用 notify Tool 通过企业微信推送："已自愈，root cause: OOM，已重启，详情见日志链接"。

全程无人值守 · 数据留在企业内网 · 每一步可审计 · 重启后 MEMORY.md 记住这次事故，下次自动参考。

OpenClaw: 个人级 Agent OS 的极致

247k

GitHub Stars

25

渠道平台

13,729

社区 Skill

设计哲学

"Your own personal AI assistant. Any OS. Any Platform." 核心用户是个人开发者和小团队，做到极致：25 个渠道（飞书/企微/WhatsApp/Discord...）+ 13729 个社区 Skill + Live Canvas + Voice Wake

技术栈

Node.js + TypeScript 实现。MIT 协议。Pete Steinberger（PSPDFKit 创始人）2025 年 11 月发布，2026 年 2 月作者加入 OpenAI，项目转入开源基金会维护

企业级空白

Cisco 评级"unacceptable cybersecurity risk"。CVE-2026-25253（1-click RCE）。身份认证、权限管理、可审计性、合规背书这些企业级硬要求都是 day one 缺失。Tank OS（Red Hat 工程师做的二次加固）是补丁，不是 day one 设计

结论

"OpenClaw 是个极其强大的应用，但配置不当会很危险" ——Sally O'Malley, Red Hat 首席工程师

```
bash - 80x24

$ /specify
  ↳ AI 追问边界条件，输出可执行规格

$ /plan
  ↳ 生成技术方案，等你 review 确认

$ /tasks
  ↳ 拆解为独立可验收任务列表

$ /implement
  ↳ 逐任务执行，每步可见可控
```

OpenClaw 证明了开源 Agent OS 的市场价值，同时也证明了"个人优先"和"企业优先"是两套不同的设计。

Hermes Agent: 企业方向的探索者

135k

GitHub Stars

18

渠道平台

3层

记忆体系

设计哲学

"The agent that grows with you" NousResearch (知名开源 AI 实验室) 2026 年 2 月发布。Python 实现。
比 OpenClaw 更强调工程级健壮性

独特机制

自我进化 Curator: v0.12.0 引入, 7 天周期自动 grade/consolidate/prune skill library。三层记忆 + Honcho 12 层 dialectic user modeling。安全扫描: memory entry 写入前检测 prompt injection

企业落地证据

PTG (Petronella Technology Group) 包装成企业管理服务, 4 万美元 + 月度 retainer。腾讯云/阿里云官方部署指南, CMMC/HIPAA/SOC 2 合规证据

企业级空白

多 Agent 编排还在 L0 (单 agent + 抛弃式 delegation)。多租户靠 HERMES_HOME 环境变量切换, 不是真正的 RBAC/SSO。审计要自补 (JSONL + SQLite)



Hermes 比 OpenClaw 更接近企业级, 但"更接近"不等于"到达"。多租户、SSO、审计、合规, 仍然是空白。

设计哲学对比：同类不同档

维度	OpenClaw	Hermes Agent
实现语言	Node.js / TypeScript	Python
定位	个人 / 小团队	小团队 → 企业（探索中）
渠道数量	25 个渠道，覆盖极广	18 个渠道
记忆体系	单层 session	三层 + Honcho user modeling
安全扫描	无内置安全扫描（CVE 真实存在）	Memory 写入前安全扫描
多租户	无	HERMES_HOME 切换（非 RBAC）
审计	无原生审计	JSONL + SQLite（需自补）
企业 IT 集成	社区扩展，无 SDK	社区 connector，无 first-class SDK
商业化样本	OpenClaw + Tank OS 二次加固	PTG 企业管理服务 4 万 + / 年

两者都验证了开源 Agent OS 的市场价值。缺的是：Java 生态 + day one 企业级治理。

两者共同留下的空白

空白 1 完整的企业级治理

多租户 RBAC、SSO 接入（企业 AD / Okta / 企业微信认证）、审计架构、合规留证——这些都是 day one 缺失，不是打补丁能解决的。

空白 2 企业 IT 系统深度集成

ERP / CRM / CMDB / 监控系统对接，要么没有，要么靠社区补丁。企业最痛的是"我要把 Agent 接进自己系统"，这件事没有 first-class 支持。

空白 3 Java 生态完全真空

OpenClaw 是 Node.js，Hermes 是 Python。Java 生态里没有任何项目把"Agent OS"作为定位。大量企业后端是 Java/Spring，Agent OS 接入成本极高。

空白 3 是最大的机会——它不是竞争，是在另一个象限里独自布局

为什么是 Java：不是偏好，是硬约束

01 Spring Boot 是企业后端事实标准

装一个 Java Agent OS = 装一个 Spring Boot 应用。IT 部门不用学新东西，运维体系无缝对接。

02 Spring AI Alibaba 已解决底层 LLM 调用

12 个主流 LLM connector (DeepSeek / 通义 / Kimi / 智谱...) 不需要重造。站在巨人肩膀上。

03 Java 工程师密度最高

企业 IT 市场上 Java 工程师密度远超 Python 和 Node.js。接入、二次开发、扩展 Tool 的人才门槛最低。

04 JVM 运维工具链成熟

Nacos / Sentinel / SkyWalking / Arthas / Prometheus + Grafana，企业现有运维体系直接复用，不用单独搭一套。

05 企业现有 Java 系统对接成本最低

ERP / CRM / CMDB 多数是 Java 接口，Tool 直接调 Java 服务，不需要写胶水代码。这是 OpenClaw 和 Hermes 用户最痛的点。

06 严监管行业的确定性选择

银行 / 政府 / 医疗 / 电信核心系统是 Java，私有部署 + 合规过审是硬要求。GraalVM Native Image 解决了 Java 启动慢的唯一顾虑。

OryxOS

基于 Java 实现的 · 面向企业场景的 · 开源可私有部署的 · Agent OS

▶ 跟 OpenClaw 的关系

同类不同档。OpenClaw 偏个人 (Node.js)，OryxOS 直接定位企业场景 (Java)。Skill 体系通过 SKILL.md 互通，生态互补不竞争。

▶ 跟 Hermes 的关系

同类不同档。Hermes 偏个人到小团队 (Python)，OryxOS day one 就考虑多租户 / SSO / 审计 / 合规。

▶ 跟 Dify 的关系

互补。Dify 做可视化 workflow 编排，OryxOS 做运行时基础设施。可以组合：Dify 作为应用层，OryxOS 作为基础设施层。

▶ 跟 Spring AI 的关系

复用。OryxOS 的 Provider 层直接基于 Spring AI Alibaba，不重复造轮子。Spring AI 是底层，OryxOS 是上层 Agent OS。

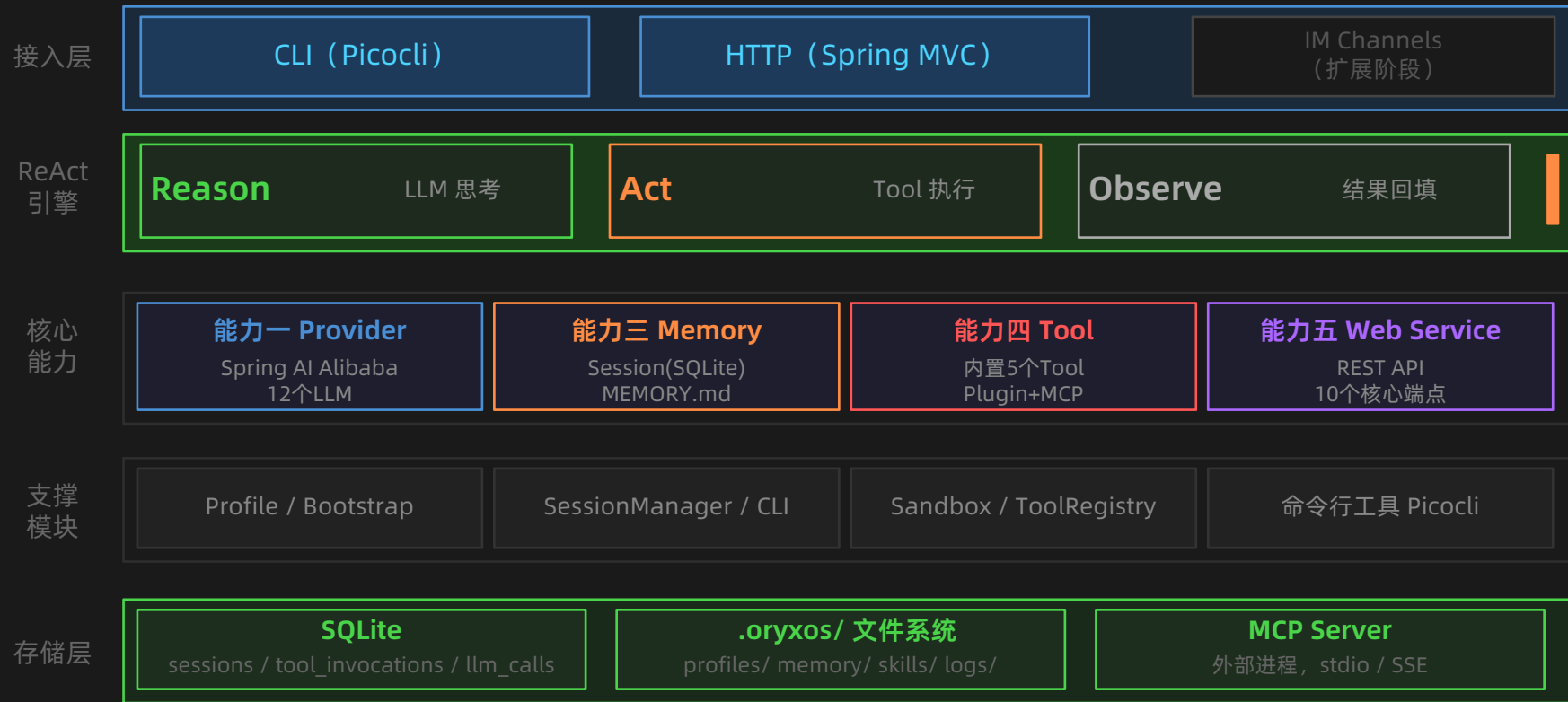
▶ 核心承诺

数据完全留在企业自己的基础设施，部署在企业自己的 K8s / 虚拟机 / 物理机上，不锁任何云生态。

▶ Apache 2.0 协议

完全开源，源代码可审计，企业可 fork 自维护，合规要求都能满足。

OryxOS 整体分层架构



OryxOS 五大核心能力

ReAct 循环是引擎。四大能力是燃料。没有任何一个，引擎跑不起来。

能力一

对接 LLM

Provider 抽象

Spring AI Alibaba 的 12 个主流 LLM connector 直接复用。DeepSeek / 通义 / Kimi / 智谱 / 豆包 / Anthropic / OpenAI...运行时切换，零 lock-in。Agent 不感知调哪家。

能力三

Memory

三层记忆

会话记忆 (SQLite 持久化) + 长期记忆 (MEMORY.md + 两个内置 Tool) + 情景记忆 (扩展阶段)。跨对话记住用户偏好、项目背景、历史决策。

能力四

Tool 体系

内置 5 + Plugin 三档

内置：文件 / Shell / HTTP / save_memory / recall_memory。Plugin：零代码 SKILL.md + MCP，轻代码自写 MCP server，重代码 @Tool 注解。主推零代码。

能力五

Web Service

REST API 对外暴露

10 个核心端点覆盖会话管理、Agent 调用、Profile/Memory/Tool 信息、系统状态。业务系统通过 HTTP 接入所有能力，跨语言集成，这是 OryxOS 区别于“个人助手”的关键。

Plugin Tool 三档接入：主推零代码

让业务方扩展 OryxOS 的能力。能用低门槛就不用高门槛。

零代码

方式一

写一份 SKILL.md 描述任务 + 复用现成 MCP server (GitHub / Slack / Notion 等)。LLM 自己读 SKILL.md，自己组合工具，自己完成任务。不写一行代码。

daily-pr-digest.md + github-mcp +
slack-mcp = 每日 PR 简报自动推送

轻代码

方式二

用任何语言自己写 MCP server，通过 MCP 协议暴露工具。OryxOS 作为 MCP Client 连接进来，把工具注册到 Tool Registry。适合接企业自有系统 (ERP / CRM / CMDB)。

python my-erp-mcp/server.py →
OryxOS 自动接入

重代码

方式三

用 Spring AI @Tool 注解标注 Java 方法，OryxOS 启动时自动扫描注册。直接调 Java 服务，复用 Spring Bean，跟 Spring Security 集成做权限控制。性能最好。

```
@Tool public String queryOrder(String  
orderId) { ... }
```

SKILL.md 兼容 agentskills.io 开放标准。OpenClaw 的 13729 个社区 Skill 理论上可以直接导入 OryxOS

零代码落地真实场景：SKILL.md

业务方写一份 markdown，不写代码，Agent 自己组合工具跑通任务

daily-pr-digest.md

```
---
name: daily-pr-digest
trigger: cron("0 9 * * *")
required_tools:
  - github-mcp
  - slack-mcp
---
```

任务说明

每天早上 9 点：

1. 用 `github-mcp.list_pull_requests` 拉取昨日所有 PR
2. 统计评审状态
`approved` / `changes_requested`
3. 用 markdown 表格汇总简报
4. 用 `slack-mcp.send_message` 推送到 `#pr-review` 频道

OryxOS 跑起来之后：

- 1 SkillLoader 加载 .md 注入 system prompt
- 2 McpClientService 连接 github-mcp + slack-mcp，工具注册到 ToolRegistry
- 3 cron 触发 Agent
- 4 LLM 读到 SKILL.md，自己决定调 `github-mcp.list_pull_requests`
- 5 拿到 PR 列表，分析状态，生成 markdown 简报
- 6 调 `slack-mcp.send_message` 推到频道

7 整个流程 0 行代码

Profile YAML 里引用 skill 和 mcp_servers，配置一份就启动一个新 Agent 场景

企业部署：数据完全私有，跟现有体系零摩擦

极客时间
训练营

.oryxos/ 工作区结构

profiles/ ← Agent 配置 YAML
 └─ default.yaml
memory/
 └─ MEMORY.md ← 长期记忆
skills/ ← SKILL.md 文件
mcp_servers.yaml ← MCP 配置
sessions/ ← SQLite 会话
logs/ ← 结构化日志
AGENTS.md ← Bootstrap
SOUL.md ← Agent 人格
USER.md ← 用户偏好
oryxos.db ← SQLite 数据库

三种运行模式

oryxos chat

交互式多轮对话。开发调试和日常使用的主要方式。`--message "xxx"` 单条消息后退出

oryxos serve

HTTP API 模式。启动后在 8080 端口开放 REST 接口，业务系统通过 HTTP 接入。Java 21 virtual thread 支撑几千并发

oryxos gateway

守护进程模式。同时服务多个 Channel（企业微信 / 飞书 / 钉钉，扩展阶段补齐）

JDK 21 + Spring Boot 3.x · 单可执行 JAR · Spring AI
Alibaba · SQLite · Picocli · 扩展阶段 GraalVM Native Image（启动 < 100ms）

装好就跑，不依赖外部服务，跟企业现有 K8s / Java 运维工具链无缝对接

3 周 12 小时，从零到 OryxOS 1.0

每周 4 小时，每周末有可演示成果。训练营最重的项目，用 Spec-Kit 驱动开发。

第一周

LLM + ReAct

4 小时

Maven 多模块骨架 / oryxos init / Profile
YAML 解析
ProviderService (DeepSeek 或 Kimi)
ReActLoop + PromptBuilder +
ToolExecutor (< 100行 Java)
一个 HTTP Tool + CLI Channel

Demo: "查天气并告诉我穿什么"
Agent 通过 ReAct 循环调 HTTP Tool 完成任务

第二周

Memory + Tool

4 小时

MemoryService (MEMORY.md 读写)
save_memory + recall_memory 内置
Tool
文件 Tool + Shell Tool (带白名单沙箱)
McpClientService (连接外部 MCP server)
SkillLoader (SKILL.md 加载注入
prompt)

Demo: "记住我用 Spring Boot" → 重启后
再问项目问题
Agent 自然引用之前记的偏好给出建议

第三周

Web Service + 工程化

4 小时

Spring MVC + virtual thread
6 个 ApiController + 10 个 REST 端点
Session 持久化到 SQLite
BootstrapLoader + 12 个 CLI 命令
结构化日志 + 项目主页 (VitePress)
每位学员提交一份社区文档 PR

Demo: 外部系统通过 REST API 完整调用
OryxOS
5 个端点联动完成一次业务流程

Spec-Kit 驱动开发：需求文档 + 技术方案直接喂给 AI

OryxOS 的所有文档（业界调研 + 需求文档 + 技术方案）直接喂给 Spec-Kit，不重写

constitution.md

6 条非协商原则

JDK 21 + Spring Boot · 自实现 ReAct · Plugin Tool 三档 · SQLite + MEMORY.md · 跑通优先于完美

spec.md

5 个 User Story

US-1 (LLM) → US-2 (ReAct) → US-3+US-4 并行 (Memory+Tool) → US-5 (Web Service)。每个 US 有验收 demo

plan.md

技术方案转成实施计划

11 个 Maven 模块划分 · 关键技术决策展开 · 模块间协作关系 · 数据流设计

tasks.md

每个 US 拆分 task

1-2 个文件每 task 颗粒度。可并行的 task 标出来。每个 task 完成有可运行代码

`/speckit.analyze` 每个 US 完成后跑一次，检查 spec 和代码一致性，发现漂移立刻修正

```
bash - 80x24

$ /specify
  ↳ AI 追问边界条件，输出可执行规格

$ /plan
  ↳ 生成技术方案，等你 review 确认

$ /tasks
  ↳ 拆解为独立可验收任务列表

$ /implement
  ↳ 逐任务执行，每步可见可控
```

第四节课会实际演示 Spec-Kit 跑通 OryxOS 代码框架，这节课是认知层的铺垫

本节总结 + 下节预告

这节课讲了

▸ Agent 五组件

LLM + Memory + Loop + Tools + Environment, 任何 Agent 都是这五件事的组合

▸ 主流 Agent OS 对比

OpenClaw (个人级极致) vs Hermes (企业方向探索), 两者留下的三个空白

▸ OryxOS 差异化

Java 实现 + 企业场景 + 私有部署, 填 Java 生态真空, day one 企业级治理

▸ 设计方案

五层架构 + 五大核心能力 + Plugin Tool 三档 + 3 周 12 小时里程碑

第四节预告

▸ 用 Spec-Kit 起手 OryxOS

从一句话需求到 spec.md + plan.md + tasks.md, 演示 Spec-Kit 完整流程

▸ constitution.md 实战

把工程原则变成 AI 必须遵守的硬约束, 6 条原则如何影响 AI 写代码

▸ Subagents 并行实现

4 个并行 subagent 同时做 4 个模块, 展示 AI 编程在复杂系统里的真实节奏

▸ 搭建开源官网

OryxOS 官网从零到可访问, 让项目对外有门面

下一节才是真正的动手实战, 这节课是技术认知的地基